Department of Computer Science and Engineering
MBM University, Jodhpur

ESRC Workshop

Internship Report Submitted in Partial Fulfilment of the Requirements for the Degree of

Bachelor of Engineering

in

Computer Science Engineering

Submitted by:

Renu Gehlot
(Roll No.
25UMITE2774)

Under the Supervision of

Mr. Alok Singh Gahlot
(Assistant Professor, CSE)

TABLE OF CONTENTS

Acknowledgement

I would like to express my sincere gratitude to **M.B.M University** for providing me with the opportunity to undergo this **45-day internship in Embedded Systems and Robotics**. The

internship offered valuable practical exposure and significantly enhanced my understanding of embedded systems, robotics, Linux, computer vision, and autonomous technologies.

I am deeply thankful to my **project mentor, faculty members, and trainers** for their constant guidance, encouragement, and technical support throughout the internship. Their expertise, valuable suggestions, and motivation played an important role in the successful completion of the training and the projects undertaken during this period.

I also extend my heartfelt thanks to **MBM University, Jodhpur**, for encouraging students to participate in industry-oriented training programs that bridge the gap between academic learning and practical application.

Finally, I would like to thank my family and friends for their continuous encouragement, support, and motivation throughout the internship. Their confidence in me inspired me to complete this training successfully.

The knowledge, practical skills, and experience gained during this internship will greatly contribute to my academic growth and future professional career in the field of Embedded Systems, Robotics, and Artificial Intelligence.

Abstract

The **45-day internship in Embedded Systems and Robotics** provided an excellent opportunity to gain practical knowledge and hands-on experience in embedded programming, robotics, Linux, and computer vision. The training focused on understanding the fundamentals of microcontrollers, sensor interfacing, communication protocols, motor control, and autonomous robotic systems through a combination of theoretical sessions and practical implementations.

During the internship, various technologies and tools such as **Arduino UNO, ESP32, Python, OpenCV, Ubuntu Linux, and Visual Studio Code** were used to develop and test embedded applications. Practical exposure included Linux command-line operations, sensor interfacing, PID control, computer vision techniques, and robotic system development.

Several mini projects were successfully completed, including the **Smart Health Guardian**, **Fast Line Follower Robot using PID Control**, **Bluetooth-Controlled RC Car**, **Aircraft Navigation using MPU6050**, and various **Computer Vision experiments** such as face detection, lane detection, object detection, and traffic light detection. The internship concluded with the development of a **Vision-Based Self-Driving Car**, integrating image processing, lane detection, and steering control to demonstrate autonomous navigation.

Overall, the internship significantly enhanced technical knowledge, programming skills, hardware interfacing capabilities, and problem-solving abilities. It provided valuable practical experience and strengthened the foundation required for future work in embedded systems, robotics, automation, and intelligent transportation systems.

CHAPTER 1

INTRODUCTION

1.1 Introduction to Embedded Systems

Definition

An **Embedded System** is a dedicated computing system designed to perform one or more specific tasks within a larger mechanical or electrical system. Unlike general-purpose computers, embedded systems are optimized for reliability, efficiency, low power consumption, and real-time operation. They are integrated into everyday products such as automobiles, medical devices, industrial automation systems, smart appliances, drones, and robotics.

Embedded systems are generally built around microcontrollers or microprocessors and are programmed to execute predefined functions. These systems often interact with sensors to collect data and actuators to perform physical actions, making them indispensable in automation and control applications.

Evolution of Embedded Systems

The development of embedded systems has progressed significantly over the past few decades:

Generation	Characteristics
First Generation	8-bit microcontrollers with limited functionality
Second Generation	16-bit controllers with improved memory and peripherals
Third Generation	32-bit ARM-based controllers with advanced processing capabilities
Modern Generation	IoT-enabled, AI-assisted, cloud-connected embedded platforms

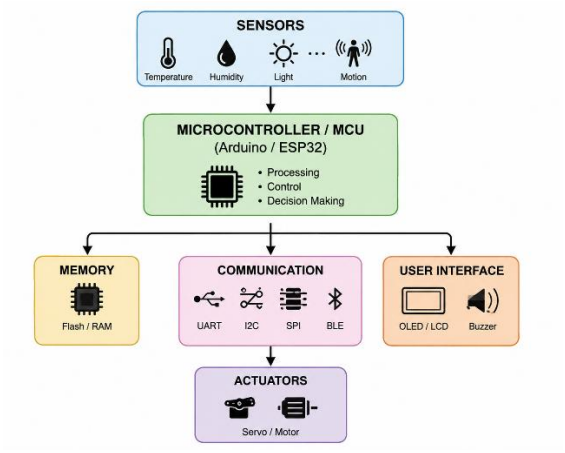
Today, embedded systems integrate technologies such as wireless communication, artificial intelligence, machine learning, and computer vision, enabling autonomous decision-making and intelligent automation.

Characteristics of Embedded Systems

Embedded systems possess several distinguishing characteristics:

- Dedicated functionality
- Real-time processing

- Low power consumption
- High reliability
- Compact size
- Cost efficiency
- High performance
- Long operational life
- Hardware-software integration



Basic Architecture of an Embedded System

1.2 Introduction to Robotics

Robotics is an interdisciplinary branch of engineering that integrates mechanical engineering, electronics, computer science, artificial intelligence, and control systems to design and develop intelligent machines capable of performing tasks autonomously or semi-autonomously.

A robotic system typically consists of sensors, actuators, controllers, communication modules, and software algorithms that enable perception, decision-making, and physical interaction with the environment.

Components of a Robot

Sensors
↓
Controller
(Microcontroller)
↓
Decision Algorithm
↓

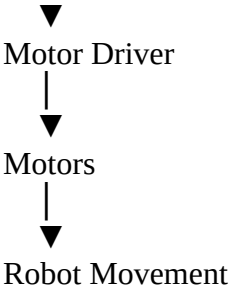


Figure 1.2: Basic Robotic Control System

Applications of Robotics

Robots are extensively used in:

- Manufacturing automation
- Warehouse logistics
- Healthcare and surgery
- Self-driving vehicles
- Space exploration
- Search and rescue
- Agriculture
- Military surveillance
- Smart factories

1.3 Internship Overview

The **45-day internship in Embedded Systems and Robotics** was designed to provide practical exposure to embedded hardware, programming, robotics, Linux, computer vision, and autonomous systems.

The internship combined theoretical sessions with hands-on laboratory experiments and project-based learning. Throughout the training, various embedded platforms, including Arduino and ESP32, were used to interface sensors, actuators, and communication modules. Students gained experience in implementing robotics algorithms, PID control, Linux command-line operations, and OpenCV-based vision systems.

The internship culminated in the development of several mini projects and a major project, enabling participants to apply their acquired knowledge to solve real-world engineering problems.

1.4 Objectives of the Internship

The primary objective of this internship was to bridge the gap between theoretical knowledge acquired during undergraduate studies and practical implementation in the field of Embedded Systems and Robotics. The internship focused on developing technical competence, problem-

solving skills, and hands-on experience with modern embedded hardware, software development, and autonomous robotic systems.

1.5 Internship Duration

Weekly Timeline

Week	Topics Covered	Practical Activities
Week 1	Introduction to Embedded Systems	Arduino Basics, Linux Installation
Week 2	Sensors & Communication Protocols	Sensor Interfacing, UART, SPI, I²C
Week 3	Robotics Fundamentals	Motor Control, PID Control
Week 4	Computer Vision	OpenCV, Image Processing
Week 5	Mini Projects	Robot Development
Week 6	Major Project	Vision-Based Self-Driving Car

1.6 Technologies Used During the Internship

A wide range of software platforms, development environments, embedded hardware, and communication technologies were utilized during the internship. Exposure to these technologies provided practical understanding of industrial embedded system development.

Software Technologies

Software	Purpose
Arduino IDE	Programming Arduino Boards
Visual Studio Code	Python Development
Python	Computer Vision
OpenCV	Image Processing
Ubuntu Linux	Development Environment
Git	Version Control

Embedded Hardware

Hardware	Application	Ultrasonic Sensor	Distance Measurement
Arduino UNO	Microcontroller Development	Servo Motor	Position Control
ESP32	IoT Applications	DC Motor	Robot Locomotion
HC-05	Bluetooth Communication	OLED Display	Information Display
MPU6050	Motion Sensing	MAX7219	Scrolling Text Display
LM35	Temperature Measurement		

1.7 Applications of Embedded Systems and Robotics

1.7.1 Healthcare and Medical Devices -Embedded systems play a crucial role in modern healthcare by enabling accurate monitoring, diagnosis, and treatment.

1.7.2 Agriculture- Smart agriculture uses embedded systems to improve crop yield while reducing resource consumption.

1.7.3 Aerospace and Defense -Embedded systems are widely used in:

1.7.4 Smart Home Automation-Embedded systems enable intelligent home automation through interconnected devices.

1.7.5 Internet of Things (IoT) -The Internet of Things connects embedded devices through the internet, enabling remote monitoring and control.

1.8 Internship Outcome

The successful completion of this internship enabled the development of several practical embedded systems and robotics applications, including:

- LED Programming
- Sensor Interfacing Experiments
- Servo Motor Control
- Bluetooth Controlled Robot
- Obstacle Avoiding Robot
- Fast Line Follower Robot using PID Control
- Aircraft Navigation using MPU6050
- Smart Health Guardian
- Vision-Based Self-Driving Car

These projects provided hands-on experience in system integration, embedded programming, robotics, and computer vision, preparing students for future industrial and research opportunities.

1.10 Future Scope

Embedded systems and robotics continue to evolve with advances in AI, edge computing, and IoT. Future developments are expected to focus on:

- AI-powered autonomous robots
- Collaborative industrial robots (Cobots)
- Edge AI devices
- Smart healthcare systems

CHAPTER 2

TECHNICAL TRAINING

Chapter Overview

This chapter describes all the theoretical concepts and practical knowledge acquired during the 45-day internship. The training covered embedded system fundamentals, programming of Arduino and ESP32 development boards, Linux operating system, communication protocols, robotics, motor control, computer vision using OpenCV, and software simulation tools.

2.1 Embedded Systems Fundamentals

Introduction

An embedded system is a specialized computer system designed to perform a dedicated function within a larger electronic or mechanical system. Unlike general-purpose computers, embedded systems are optimized for specific applications, offering high reliability, low power consumption, compact size, and real-time performance.

Today, embedded systems are found in almost every modern electronic device, including smartphones, automobiles, medical equipment, industrial robots, drones, household appliances, and aerospace systems.

Embedded systems continuously monitor environmental conditions through sensors, process the collected data using a microcontroller or processor, and generate appropriate outputs through actuators or displays.

Components of an Embedded System

An embedded system generally consists of the following hardware and software components:

1. Input Devices - Input devices collect information from the surrounding environment.

- Temperature Sensor (LM35)
- Ultrasonic Sensor
- IR Sensor
- MPU6050 Gyroscope
- Camera Module
- Push Buttons

2. Processing Uni - The processing unit performs all computations and controls the system.

- Arduino UNO
- ESP32
- STM32

- Raspberry Pi Pico
- ARM Cortex-M Controllers

.**3. Memory** -Memory stores the program instructions and temporary data.

Types of memory include:

Memory	Purpose
Flash Memory	Stores program permanently
EEPROM	Stores configuration data
SRAM	Temporary data during execution

4. Output Devices - Output devices respond according to the processed data.

Examples include:

- LEDs
- LCD Displays
- OLED Displays
- Servo Motors
- DC Motors
- Buzzers
- Relays

Embedded System Architecture

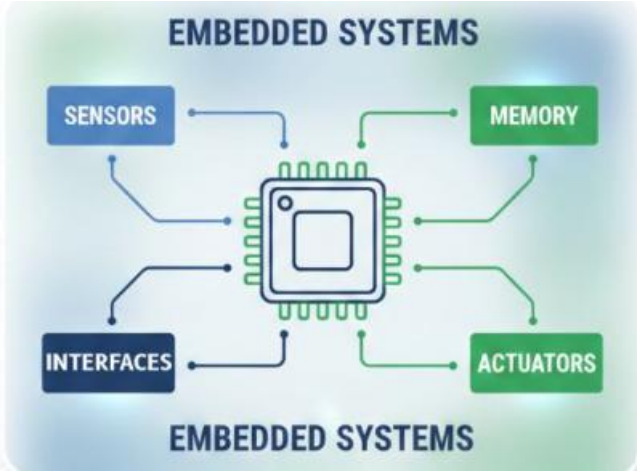


Figure 2.1: General Architecture of an Embedded System

2.2 Arduino Programming

Introduction

Arduino is an open-source embedded development platform that simplifies microcontroller programming. It provides an integrated development environment (IDE) and a wide range of compatible hardware boards, making it one of the most popular platforms for learning embedded systems and rapid prototyping.

During the internship, Arduino UNO was used to interface sensors, actuators, displays, and communication modules. Students developed programs to control LEDs, read sensor values, operate motors, and implement basic robotics applications.

Features of Arduino UNO

Feature	Specification
Microcontroller	ATmega328P
Operating Voltage	5V
Digital I/O Pins	14
PWM Pins	6
Analog Inputs	6
Flash Memory	32 KB
SRAM	2 KB
EEPROM	1 KB
Clock Speed	16 MHz

Arduino IDE

The Arduino Integrated Development Environment (IDE) is used to write, compile, and upload programs to Arduino boards. It provides a simple interface with built-in libraries that simplify hardware programming.

Main Components

- Code Editor
- Verify Button
- Upload Button
- Serial Monitor
- Library Manager
- Board Manager

Structure of an Arduino Program

Every Arduino sketch follows a standard structure:

setup(): The setup() function runs once when the board is powered on or reset. It is used to initialize pins, communication interfaces, and libraries.

loop(): The loop() function runs continuously after setup(). It contains the main program logic, such as reading sensor values, controlling outputs, and making decisions.

Digital Input and Output - Digital pins can operate in two states:

- HIGH (Logic 1)
- LOW (Logic 0)

They are commonly used to:

- Turn LEDs ON/OFF
- Read push-button states
- Control relays
- Drive digital modules

Analog Input - Analog pins measure continuously varying voltages and convert them into digital values using the Analog-to-Digital Converter (ADC).

Common analog sensors:

- LM35 Temperature Sensor
- Potentiometer
- Light Sensor (LDR)
- Soil Moisture Sensor

PWM (Pulse Width Modulation) - PWM is used to simulate analog output by rapidly switching a digital pin ON and OFF.

Applications include:

- Motor speed control
- LED brightness control
- Servo positioning

Serial Communication - Arduino communicates with computers and other devices through serial communication.

Common baud rates:

- 9600 bps
- 115200 bps

Applications:

- Sensor monitoring
- Debugging
- Bluetooth communication
- ESP32 communication

2.3 ESP32 Programming

Introduction

The **ESP32** is a low-cost, high-performance System-on-Chip (SoC) developed by Espressif Systems. It is one of the most widely used microcontrollers for Internet of Things (IoT), robotics, automation, wearable electronics, and wireless communication applications due to its integrated Wi-Fi and Bluetooth capabilities.

Compared to Arduino UNO, ESP32 offers significantly higher processing power, larger memory, multiple communication interfaces, and dual-core architecture. These features make it suitable for advanced embedded applications requiring real-time data processing and wireless connectivity.

During the internship, ESP32 was used for developing IoT applications, wireless communication projects, sensor monitoring, and robotic control systems.

Features of ESP32

Feature	Specification
Processor	Dual-Core Xtensa LX6
Clock Speed	Up to 240 MHz
SRAM	520 KB
Flash Memory	4 MB (Typical)
Wi-Fi	802.11 b/g/n
Bluetooth	BLE & Classic

Feature	Specification
GPIO Pins	Up to 34
ADC Channels	18
DAC Channels	2
PWM Channels	16
UART	3
SPI	4
I²C	2

Advantages of ESP32

- Built-in Wi-Fi
- Built-in Bluetooth
- High processing speed
- Large number of GPIO pins
- Low power consumption
- Multiple communication interfaces
- Suitable for IoT applications
- Cost-effective
- Real-time performance

Applications of ESP32

ESP32 is widely used in:

- Smart Home Automation
- Smart Agriculture
- Industrial Monitoring
- Weather Stations
- Wearable Devices
- Smart Health Monitoring
- Autonomous Robots
- Wireless Sensor Networks
- Security Systems

2.4 Sensors and Actuators

Introduction

Sensors and actuators form the foundation of every embedded system. A sensor collects information from the physical environment, whereas an actuator converts electrical signals into physical movement or action.

Embedded systems continuously acquire sensor data, process it using a microcontroller, and generate output through actuators.

Types of Sensors Used

LM35 Temperature Sensor - The LM35 is an analog temperature sensor that produces an output voltage proportional to temperature in degrees Celsius.

Ultrasonic Sensor -The ultrasonic sensor measures distance by transmitting ultrasonic waves and calculating the time taken for the reflected wave to return.

MPU6050 - The MPU6050 is a six-axis motion sensor integrating a 3-axis accelerometer and a 3-axis gyroscope.

It measures:

- Pitch
- Roll
- Yaw
- Acceleration
- Angular velocity

IR Sensor -Infrared sensors detect reflected infrared light.

Camera Module -The camera acts as the primary sensor in computer vision systems.

Actuators

Servo Motor -Servo motors provide accurate angular position control.

Applications:

- Robotic Arm
- Steering Mechanism
- Camera Gimbal
- Automated Gates

DC Motor -DC motors convert electrical energy into rotational motion.

Applications:

- Robot Movement
- Conveyor Systems
- Electric Vehicles
- Cooling Fans

Relay -Relays electrically isolate low-power control circuits from high-power loads.

2.5 Communication Protocols

Modern embedded systems require efficient communication between controllers, sensors, displays, and external devices.

UART - Universal Asynchronous Receiver Transmitter (UART) is one of the simplest serial communication protocols.

Characteristics:

- Two-wire communication
- Full duplex
- Asynchronous
- No clock line

Applications:

- Arduino ↔ Computer
- ESP32 ↔ Bluetooth Module
- GPS Communication

SPI - Serial Peripheral Interface (SPI) provides high-speed communication

Applications:

- OLED Display
- SD Card
- LCD
- Sensors

I²C -Inter-Integrated Circuit (I²C) is a synchronous communication protocol.

Characteristics:

- Two-wire communication
- Multiple devices
- Address-based communication
- Low wiring complexity

Applications:

- MPU6050
- OLED Display
- RTC Module

Comparison of Communication Protocols

Feature	UART	SPI	I²C
Clock	No	Yes	Yes
Speed	Medium	High	Medium
Wires	2	4	2
Multiple Devices	Limited	Yes	Yes
Complexity	Low	Medium	Medium

2.6 Motor Control

Motors convert electrical energy into mechanical motion and are fundamental components in robotics.

Types

- DC Motor
- Servo Motor
- Stepper Motor
- Brushless DC Motor (BLDC)

Motor Driver - Microcontrollers cannot directly drive motors due to current limitations. Motor driver circuits amplify the control signals to supply sufficient current to the motor.

Common motor drivers:

- L298N
- L293D
- BTS7960

Speed Control - Motor speed is controlled using Pulse Width Modulation (PWM).

Advantages:

- Energy efficient
- Smooth control
- Precise adjustment

Direction Control -The direction of rotation is changed by reversing the polarity supplied to the motor.

Motor drivers use H-Bridge circuits for this purpose.

2.7 PID Control

Introduction

PID (Proportional–Integral–Derivative) control is one of the most widely used feedback control algorithms in industrial automation and robotics. It continuously calculates the error between the desired setpoint and the actual system output, then adjusts the control signal to minimize that error. During the internship, PID control was implemented in the **Fast Line Follower Robot** to achieve smooth and accurate path tracking.

Components of PID

Proportional (P) - Provides a correction proportional to the current error. A larger error results in a stronger corrective action.

Integral (I) -Accumulates past errors to eliminate steady-state offset.

Derivative (D) -Predicts future error by considering the rate of change, helping to reduce overshoot and oscillations.

Applications of PID

- Line Following Robot
- Self-Balancing Robot
- Drone Stabilization
- Temperature Control
- Motor Speed Control
- Autonomous Vehicles

2.8 Computer Vision and OpenCV

Introduction

Computer Vision enables machines to interpret and analyze visual information from images and videos. OpenCV (Open Source Computer Vision Library) provides a comprehensive collection of algorithms for image processing, feature extraction, object detection, and machine learning.

During the internship, OpenCV was used to develop applications such as lane detection, color detection, traffic light recognition, face detection, and object tracking.

Common OpenCV Operations

- Image resizing
- Color space conversion (RGB to HSV)
- Thresholding
- Edge detection
- Contour detection
- Morphological operations
- Face detection
- ArUco marker detection
- Lane detection

2.9 Software and Simulation Tools

The internship utilized various software tools for programming, debugging, simulation, and computer vision.

Software	Purpose
Arduino IDE	Programming Arduino boards
Visual Studio Code	Python and embedded development
Python	Programming language
OpenCV	Computer vision
Ubuntu Linux	Development environment
Git	Version control
Webots (if used)	Robot simulation
PlatformIO (optional)	Embedded development

CHAPTER 3

Linux Commands & Terminal Practice

3.1 Introduction

Linux is an open-source operating system widely used in embedded systems, robotics, IoT devices, cloud computing, and software development. It offers a powerful command-line interface (CLI) that enables users to efficiently manage files, directories, processes, networks, software packages, and system resources.

During the internship, Linux was used extensively for programming, compiling applications, managing embedded projects, and interacting with development tools such as Arduino IDE, Python, OpenCV, and Git. Familiarity with Linux commands is essential for embedded systems engineers, as many development boards (such as Raspberry Pi, NVIDIA Jetson, and BeagleBone) operate on Linux-based operating systems.

This chapter documents the Linux commands practiced during the internship, categorized according to their functionality.

Commands-

3.1 File & Directory Management

Command 1: pwd

Syntax: pwd

Description: Displays the absolute path of the current working directory.

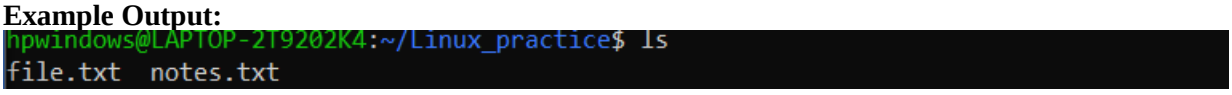
Example Output:

Command 2: ls

Syntax: ls [options][directory]

Description: Lists all files and directories in the current directory.

Example Output:



Command 3 : mkdir

Syntax: mkdir <directory_name>

Description: Creates a new directory with the specified name.

Example Output:

```
hpwindows@LAPTOP-2T9202K4:~$ mkdir Linux_practice
hpwindows@LAPTOP-2T9202K4:~$ cd Linux_practice
hpwindows@LAPTOP-2T9202K4:~/Linux_practice$ pwd
/home/hpwindows/Linux_practice
```

Command 4: ls -l

Syntax: ls -l

Description: Displays a detailed list of files, including permissions, owner, size, and modification date.

Example Output:

```
hpwindows@LAPTOP-2T9202K4:~/Linux_practice$ ls -l
total 0
-rw-r--r-- 1 hpwindows hpwindows 0 Aug  3 13:02 file.txt
-rw-r--r-- 1 hpwindows hpwindows 0 Aug  3 13:02 notes.txt
```

Command 5: ls -a

Syntax: ls -a

Description: Displays all files, including hidden files that begin with a dot (.).

Example Output:

```
hpwindows@LAPTOP-2T9202K4:~/Linux_practice$ ls -a
.  ..  file.txt  notes.txt
```

Command 6: cd

Syntax: cd <directory_name>

Description: Changes the current working directory.

Example Output:

```
hpwindows@LAPTOP-2T9202K4:~$ cd project
hpwindows@LAPTOP-2T9202K4:~/project$ pwd
/home/hpwindows/project
```

Command 7: cd ..

Syntax: cd ..

Description Moves to the parent directory.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:~/project$ cd ..
hpwindows@LAPTOP-2T9202K4:~$ pwd
/home/hpwindows
```

Command 8: cd ~

Syntax cd ~

Description Navigates to the user's home directory.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:~$ cd ~
hpwindows@LAPTOP-2T9202K4:~$ pwd
/home/hpwindows
```

Command 9: rmdir

Syntax : rmdir <directory_name>

Description : Removes an empty directory.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:~$ rmdir project
hpwindows@LAPTOP-2T9202K4:~$ ls
Linux_practice  backup.txt
```

Command 10: touch

Syntax: touch <file_name>

Description: Creates a new empty file or updates the timestamp of an existing file.

Example Output:

```
hpwindows@LAPTOP-2T9202K4:~/Linux_practice$ touch file.txt
hpwindows@LAPTOP-2T9202K4:~/Linux_practice$ touch notes.txt
```

Command 11: cat

Syntax: cat <file_name>

Description: Displays the contents of a text file on the terminal.

Example Output:

```
hpwindows@LAPTOP-2T9202K4:~$ cat file.txt
hello
```

Command 12: cp

Syntax: cp <source_file> <destination_file>

Description: Copies a file or directory from one location to another.

Example Output:

```
hpwindows@LAPTOP-2T9202K4:~$ cp file.txt copy.txt
hpwindows@LAPTOP-2T9202K4:~$ ls
Linux_practice  copy.txt  file.txt
hpwindows@LAPTOP-2T9202K4:~$ cat copy.txt
hello
```

Command13: mv

Syntax: mv <source> <destination>

Description: Moves or renames a file or directory.

Example Output:

```
hpwindows@LAPTOP-2T9202K4:~$ mv copy.txt backup.txt
hpwindows@LAPTOP-2T9202K4:~$ ls
Linux_practice  backup.txt  file.txt
```

Command 14: rm

Syntax :rm <file_name>

Description: Deletes the specified file.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:~$ rm file.txt
hpwindows@LAPTOP-2T9202K4:~$ ls
Linux_practice  backup.txt  project
```

Command 15: rm -r

Syntax : rm -r <directory_name>

Description : Deletes a directory and all its contents recursively.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:~$ rm -r Linux_practice
hpwindows@LAPTOP-2T9202K4:~$ pwd
/home/hpwindows
hpwindows@LAPTOP-2T9202K4:~$ ls
backup.txt  notes.txt
```

Command 16: find

Syntax : find <path> -name "<file_name>"

Description : Searches for files or directories by name.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:~$ find . -name "notes.txt"
./Linux_practice/notes.txt
./notes.txt
```

Command 17: realpath

Syntax : locrealpath tsetfolder

Description : Displays the absolute (complete) path of a file or directory by resolving all symbolic links and relative path components.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ realpath testfolder
/mnt/c/Windows/System32/testfolder
```

Command 18: tree

Syntax : tree

Description : Displays the directory structure in a tree format.

Expected Output

Command 19: stat

Syntax : stat <file_name>

Description : Displays detailed information about a file or directory.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:~$ stat notes.txt
  File: notes.txt
  size: 0               Blocks: 0          IO Block: 4096   regular empty file
```

Command 20: file

Syntax : file <file_name>

Description : Identifies the type of the specified file.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:~$ file notes.txt
notes.txt: empty
```

Command 21: basename

Syntax : basename <path>

Description : Extracts the filename from a complete file path.

Expected Output

Command 22: echo "Hello Linux"

Syntax: echo [text]

Description: The echo command displays the specified text or string on the terminal screen. It is also commonly used to write text into files.

Example Output:

```
hpwindows@LAPTOP-2T9202K4:~$ echo "hello" > file.txt
hpwindows@LAPTOP-2T9202K4:~$ cat file.txt
hello
```

3.2 File Viewing & Editing

Command 23: head

Syntax: head <file_name>

Description: Displays the first 10 lines of a file.

Example Output:

```
hpwindows@LAPTOP-2T9202K4:~$ head notes.txt
hello
```

Command 24: head -5

Syntax : head -5 <file_name>

Description : Displays the first five lines of a file.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:~$ head -5 notes.txt
hello
```

Command 25: tail

Syntax: tail <file_name>

Description: Displays the last 10 lines of a file.

Example Output:

```
hpwindows@LAPTOP-2T9202K4:~$ tail notes.txt
hello
```

Command 26: tail -5

Syntax: tail -5 <file_name>

Description : Displays the last five lines of a file.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:~$ tail -5 notes.txt
hello
```

Command 27: less

Syntax :less <file_name>

Description :Opens a file for viewing one page at a time.

Expected Output

```
hpwindows@LAPTOP-2T9202K4: /mnt/c/Windows/System32
apple
banana
```

Command 28: more

Syntax : more <file_name>

Description: Displays the contents of a file page by page.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ more fruit.txt
orange
apple
banana
```

Command 29: nano

Syntax: nano <file_name>

Description: Opens the Nano text editor to create or edit a file.

Expected Output

```
 GNU nano 8.7.1                          fruit.txt
orange
apple
banana
```

Command 30: vim

Syntax: vim <file_name>

Description: Opens the Vim text editor for editing files.

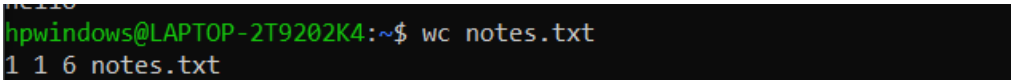
Expected Output

Command 31: wc

Syntax: wc<file_name>

Description: Counts the number of lines, words, and characters in a file.

Example Output



Command 32: nl

Syntax: nl <file_name>

Description: Displays the contents of a file with line numbers.

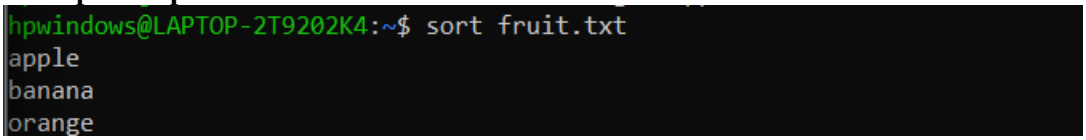
Example Output

Command 33: sort

Syntax: sort <file_name>

Description: Sorts the contents of a file in alphabetical order.

Example Output



Command 34: tac

Syntax: tac<file_name>

Description: Displays the contents of a file in **reverse order**, printing the last line first and the first line last. (tac is cat spelled backwards.)

Example Output

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ tac fruit.txt
banana
apple
orange
```

Command 35: uniq

Syntax: uniq<file_name>

Description Displays only the unique (non-repeated) consecutive lines from a sorted file by removing duplicate adjacent lines.

Example Output

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ uniq fruit.txt
orange
apple
banana
```

Command 36: diff

Syntax : diff <file name> <file_name>

Description : Compares two files line by line and displays the differences between them. If the files are identical, no output is displayed.

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ diff fruit.txt fruits.txt
3d2
< banana
```

3.3 File Permissions & Ownership

Command 37: chmod

Syntax : chmod <permissions> <file_name>

Description : Changes the access permissions of a file or directory.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ chmod u+x test.sh
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ ls -l test.sh
-rwxrwxrwx 1 hpwindows hpwindows 0 Aug  3 15:05 test.sh
```

Command 38: chmod +x

Syntax : chmod +x <file_name>

Description :Makes a file executable.

Expected Output

```
-r-xr-xr-x 2 hpwindows hpwindows     53248 Jun 21  2025 ztrace_maps.dll
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ chmod +x fruit.txt
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ ls -l
```

Command 39: chown -w

Syntax : chown –w<file name>

Description : Remove execute permission**Expected Output**

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ chmod -w test.sh
chmod: test.sh: new permissions are r-xrwxrwx, not r-xr-xr-x
```

Command 40: chgrp

Syntax :chgrp <group> <file_name>

Description : Changes the group ownership of a file.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ chgrp hpwindows fruit.txt
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$
```

Command 41: umask

Syntax : umask

Description : Displays or sets the default file permission mask.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ umask
0022
```

Command 42: whoami

Syntax : whoami

Description : Displays the username of the currently logged-in user.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ whoami
hpwindows
```

Command 43 id

Syntax :id

Description : Displays the user ID, group ID, and associated groups.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ id
uid=1000(hpwindows) gid=1000(hpwindows) groups=1000(hpwindows),4(adm),24(cdrom),27(sudo),30(dip),46(plug
dev),100(users)
```

Command 44: group

Syntax : groups

Description : Displays the groups to which the current user belongs.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ groups
hpwindows adm cdrom sudo dip plugdev users
```

Command 45: ls -l

Syntax : ls -l

Description :Displays file permissions and ownership information.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ ls -l
```

Command 46: stat

Syntax : stat <file_name>

Description :Displays detailed information about a file, including permissions, owner, and timestamps.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:~$ stat notes.txt
  File: notes.txt
  size: 0               Blocks: 0          IO Block: 4096   regular empty file
```

3.4 System Information & Monitoring

Command 47: uname

Syntax : uname

Description : Displays the name of the operating system.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ uname
Linux
```

Command 48: uname -a

Syntax:uname -a

Description :Displays detailed system information including kernel version, architecture, and hostname.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ uname -a
Linux LAPTOP-2T9202K4 6.18.33.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun 18 21:54:43 UTC 2
026 x86_64 GNU/Linux
```

Command 49: hostname

Syntax :hostname

Description :Displays the hostname of the current system.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ hostname
LAPTOP-2T9202K4
```

Command 50: date

Syntax:date

Description:Displays the current system date and time.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ date
Mon Aug  3 13:47:17 UTC 2026
```

Command51 : hostnamectl

Syntax:hostnamectl

Description: Displays and manages the system hostname and other related system information, such as the operating system, kernel version, architecture, and machine ID.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ hostnamectl
 Static hostname: LAPTOP-2T9202K4
       Icon name: computer-container
```

Command 52: uptime

Syntax:uptime

Description:Displays the system uptime and current load average.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ uptime
 13:47:36 up 56 min,  1 user,  load average: 0.09, 0.16, 0.10
```

Command 53: free -h

Syntax: free -h

Description : Displays memory usage in a human-readable format.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ free -h
               total        used        free      shared  buff/cache   available
Mem:           3.5Gi       501Mi       2.8Gi       3.5Mi       285Mi       3.0Gi
```

Command 54: df -h

Syntax :df -h

Description : Displays disk space usage of all mounted file systems.

Expected Outputjpc

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ df -h
Filesystem      Size  Used Avail Use% Mounted on
none            1.8G     0  1.8G   0% /usr/lib/modules/6.18.33.2-microsoft-standard-WSL2
none            1.8G  4.0K  1.8G   1% /mnt/wsl
```

Command 55: free

Syntax : free

Description : Displays the amount of free, used, and total physical and swap memory in the system.

Expected Output

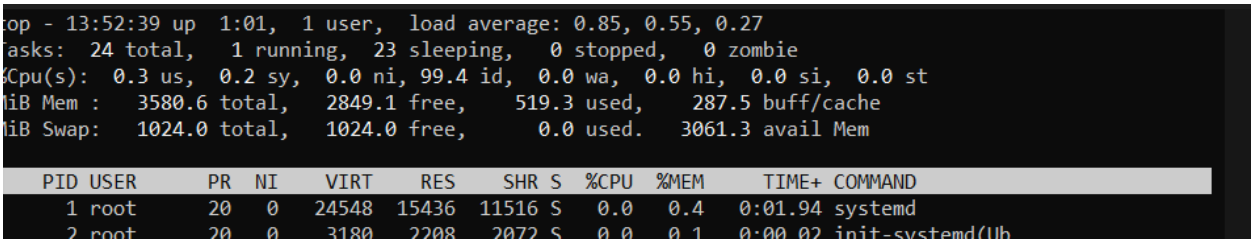
```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ free
               total        used        free      shared  buff/cache   available
Mem:         3666580      547020     2904380        3628      298592     3119560
Swap:        1048576           0     1048576
```

Command 56: top

Syntax :top

Description :Displays real-time information about system processes and CPU usage.

Expected Output

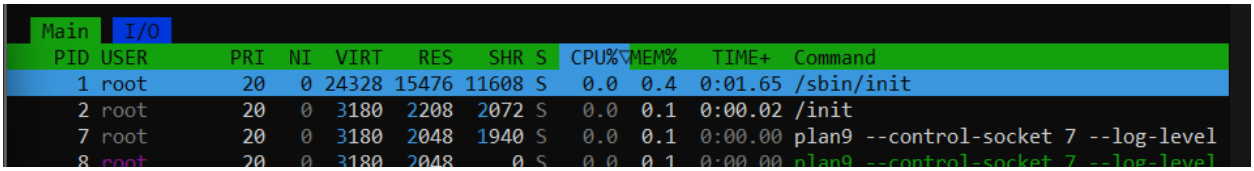


Command 57: htop

Syntax :htop

Description :Displays an interactive process monitoring interface.

Expected Output

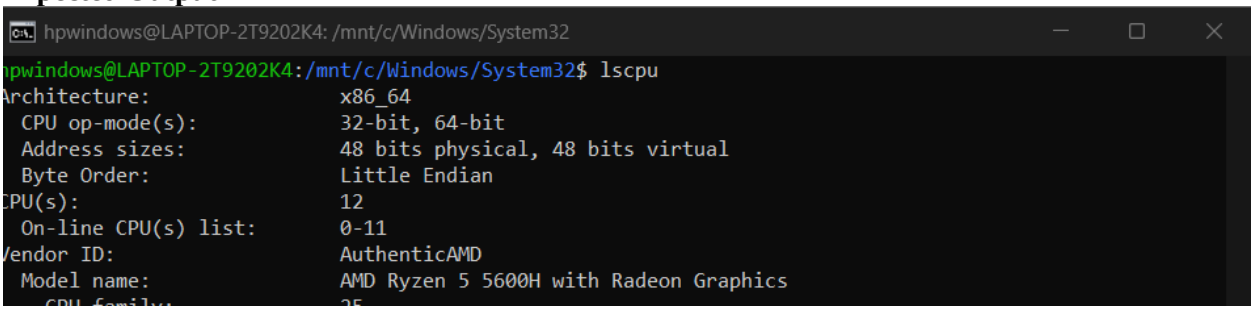


Command 58: lscpu

Syntax :lscpu

Description : Displays detailed information about the CPU architecture.

Expected Output

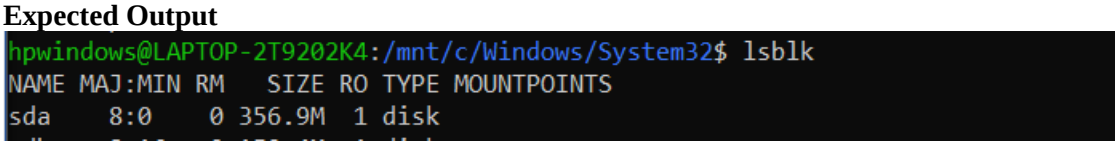


Command 59: lsblk

Syntax :lsblk

Description :Displays information about block storage devices.

Expected Output

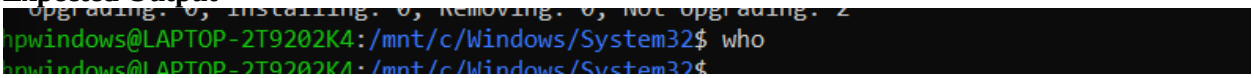


Command 60: who

Syntax : who

Description :Displays the users currently logged into the system.

Expected Output

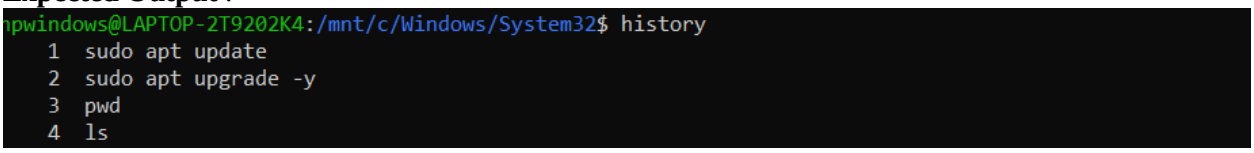


Command 61: history

Syntax : history

Description : Displays the list of previously executed commands.

Expected Output :

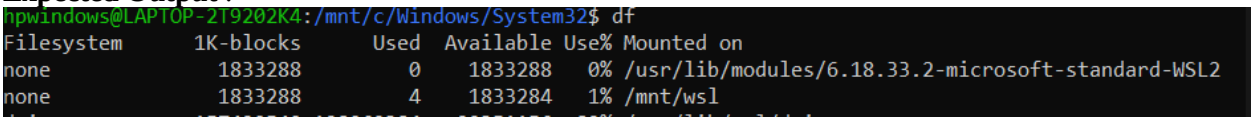


Command 62: df

Syntax : df

Description : Displays the amount of disk space used and available on file systems.

Expected Output :



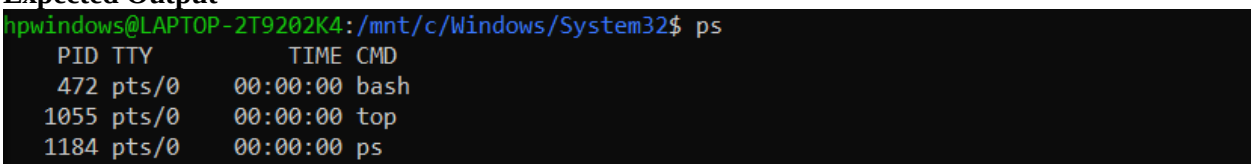
3.5 Process Management

Command 63: ps

Syntax :ps

Description :Displays the processes running in the current terminal session.

Expected Output



Command 64: ps -ef

Syntax : ps -ef

Description :Displays detailed information about all running processes.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ ps -ef
UID          PID    PPID  C STIME TTY          TIME CMD
root           1       0  0 13:36 ?        00:00:01 /sbin/init
root           2       1  0 13:36 hvc0     00:00:00 /init
root           7       2  0 13:36 hvc0     00:00:00 plan9 --control-socket 7 --log-level 4 --server-fd 8
root          50       1  0 13:36 ?        00:00:00 /usr/lib/systemd/systemd-journald
```

Command 65: ps aux

Syntax :ps aux

Description : Displays detailed information about **all running processes** on the system, including the user, Process ID (PID), CPU usage, memory usage, and the command that started each process..

Expected Output

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ ps aux
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  4.6  0.4  24328 15476 ?        Ss   15:36   0:01 /sbin/init
root           2  0.0  0.0   3180  2208 hvc0     Sl+  15:36   0:00 /init
root           7  0.0  0.0   3180  2048 hvc0     Sl+  15:36   0:00 plan9 --control-socket 7 --log-level
4 --server-fd 8
```

Command 66: kill

Syntax: kill <PID>

Description :Terminates a process using its Process ID (PID).

Expected Output

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ kill 1
-bash: kill: (1) - Operation not permitted
```

Command 67: killall

Syntax : killall <process_name>

Description : Terminates all running processes with the specified name.

Expected Output

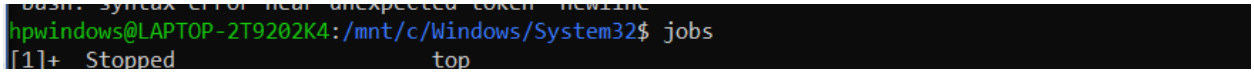
```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ killall
Usage: killall [OPTION]... [--] NAME...
       killall -l, --list
       killall -V, --version
```

Command 68: jobs

Syntax :jobs

Description : Displays the background jobs running in the current shell.

Expected Output

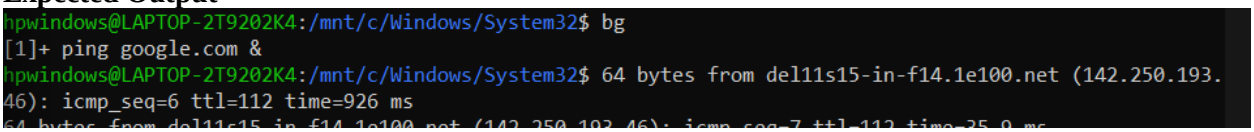


Command 69: bg

Syntax : bg %<job_number>

Description : Resumes a suspended job in the background.

Expected Output

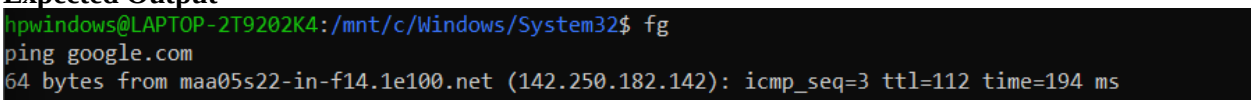


Command 70: fg

Syntax : fg %<job_number>

Description :Brings a background job to the foreground.

Expected Output

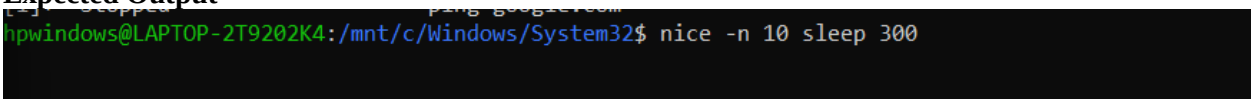


Command 71: nice

Syntax : nice -n <priority> <command>

Description : Runs a command with a specified scheduling priority.

Expected Output



Command 72: renice

Syntax : renice <priority> -p <PID>

Description : Changes the priority of a running process.

Expected Output-

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ renice 10 -p 472
renice: failed to get priority for 472 (process ID): No such process
```

Command 73: sleep

Syntax : sleep <seconds>

Description : Pauses command execution for a specified number of seconds.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ sleep 2
```

3.6 Networking Commands

Command 74: ping

Syntax : ping <hostname>

Description :Checks network connectivity by sending ICMP packets to a remote host.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ ping google.com
PING google.com (142.250.193.46) 56(84) bytes of data.
64 bytes from del11s15-in-f14.1e100.net (142.250.193.46): icmp_seq=1 ttl=111 time=55.3 ms
```

Command 75: ifconfig

Syntax :ifconfig

Description :Displays or configures network interface settings.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 172.30.161.188  netmask 255.255.240.0  broadcast 172.30.175.255
        inet6 fe80::215:5dff:fed0:feff  prefixlen 64  scopeid 0x20<link>
```

Command 76: ip addr

Syntax :ip addr

Description :Displays IP addresses and network interface information.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
```

Command 77: netstat

Syntax :netstat -tuln

Description : Displays active network connections and listening ports.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ netstat
Active Internet connections (w/o servers)
```

Command 78: hostname -I

Syntax: hostname -I

Description :Displays the IP address assigned to the system.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ hostname -I
172.30.161.188
```

Command 79: ss

Syntax : ss -tulnsud

Description :Displays socket and network connection statistics.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ ss -tulnsud
Total: 210
TCP:   3 (estab 0, closed 0, orphaned 0, timewait 0)
```

3.7 Package Management

Command 80: apt update

Syntax : sudo apt update

Description :Updates the package repository information.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ sudo apt update
[sudo: authenticate] Password:
Hit:1 http://archive.ubuntu.com/ubuntu resolute InRelease
```

Command 81: apt upgrade

Syntax : sudo apt upgrade

Description :Upgrades installed software packages.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ sudo apt upgrade
Not upgrading yet due to phasing:
  python3-software-properties  software-properties-common

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 2
```

Command 82: apt install

Syntax : sudo apt install <package_name>

Description :Installs a software package.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ sudo apt install tree -y
Installing:
  tree

Summary:
  Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 2
  Download size: 53.5 kB
  Space needed: 125 kB / 1024 GB available
```

Command 83: apt remove

Syntax : sudo apt remove <package_name>

Description ;Removes an installed software package.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ sudo apt remove tree -y
REMOVING:
  tree

Summary:
  Upgrading: 0, Installing: 0, Removing: 1, Not Upgrading: 2
  Freed space: 125 kB
```

Command 84: apt autoremove

Syntax : sudo apt autoremove

Description :Removes unused dependency packages.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ sudo apt autoremove
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 2
```

Command 85: dpkg -l

Syntax : dpkg -l

Description : Lists all installed packages.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ dpkg -l
+-- Desired=Unknown/Install/Remove/Purge/Hold
|+- Status=Not/Inst/Conf-files/Unpacked/halF-conf/Half-inst/trig-aWait/Trig-pend
||+ Err?=(none)/Reinst-required (Status,Err: uppercase=bad)
||| Name                          Version                                  Architecture Descrip
+++-==============================-========================================-============-=======
ii  adduser                       3.153ubuntu1                             all          add and
```

Command 86: snap list

Syntax : snap list

Description : Displays installed Snap packages.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ snap list
No snaps are installed yet. Try 'snap install hello-world'.
```

Command 87: snap install

Syntax : sudo snap install <package_name>

Description : Installs a package using the Snap package manager.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ sudo snap install hello-world
2026-08-03T14:13:07Z INFO Waiting for automatic snapd restart...
```

3.8 User & Group Management

Command 88: whoami

Syntax : whoami

Description :Displays the username of the current user.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ whoami
hpwindows
```

Command 89: adduser

Syntax :sudo adduser <username>

Description : Creates a new user account.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ sudo adduser student
New password:
```

Command 90: passwd

Syntax :passwd

Description :Changes the password of the current user.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ passwd
Changing password for hpwindows.
Current password:
```

Command 91: su

Syntax : su <username>

Description
Switches to another user account.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ su
Password:
```

Command 92: sudo

Syntax : sudo <command>

Description : Executes a command with administrator privileges.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ sudo apt update
[sudo: authenticate] Password:
Hit:1 http://archive.ubuntu.com/ubuntu resolute InRelease
```

Command 93: groups

Syntax :groups

Description :Displays the groups associated with the current user.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ groups
hpwindows adm cdrom sudo dip plugdev users
```

Command 94: who

Syntax :who

Description :Displays all currently logged-in users.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ who
```

3.9 Archiving & Compression

Command 95: gzip

Syntax :gzip <file_name>

Description : Compresses a file using the GZIP format.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ gzip fruit.txt
```

Command 96: gunzip

Syntax :gunzip <file_name>.gz

Description :Decompresses a GZIP-compressed file.

Expected Output

```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ gunzip fruit.txt.gz
```

Command 97: echo

Syntax :echo "text"

Description :Displays text or variables on the terminal.

Epected Output

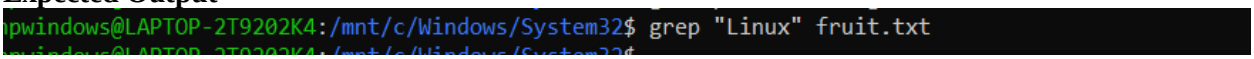
```
hpwindows@LAPTOP-2T9202K4:/mnt/c/Windows/System32$ echo "hello"
hello
```

Command 98: grep

Syntax :grep "<pattern>" <file_name>

Description:Searches for a specified pattern within a file.

Expected Output



Command 99: clear

Syntax :clear

Description : Clears all the text from the terminal screen, providing a clean workspace. It does not delete any files or command history.

Command 100: exit

Syntax :exit

Description : Closes the current terminal session or logs out of the current shell. In WSL, it exits the Ubuntu terminal and returns to Windows.

CHAPTER 4

Practical Implementations & Mini Projects

Chapter Overview

The internship emphasized practical implementation to strengthen the understanding of embedded systems, robotics, sensor interfacing, and computer vision. Throughout the training, several mini projects were designed and developed using Arduino UNO, ESP32, sensors, motors, communication modules, and OpenCV. These projects helped bridge the gap between theoretical concepts and real-world engineering applications by focusing on system design, programming, debugging, testing, and performance evaluation.

4.1 Mini Project 1: Smart Health Guardian – IoT-Based Health Monitoring Armband

4.1.1 Introduction

The **Smart Health Guardian** is a wearable embedded system designed to monitor an individual's health and provide emergency assistance. The system integrates multiple sensors with an Arduino UNO to continuously monitor environmental and physiological parameters. It aims to enhance personal safety, especially for elderly individuals, patients, and people requiring continuous health supervision.

4.1.2 Problem Statement

Many elderly people and patients live alone without constant medical supervision. Delays in detecting falls, abnormal temperature, or emergency situations can lead to serious health risks. Commercial health monitoring devices are often expensive and inaccessible to everyone.

The Smart Health Guardian addresses this issue by providing a low-cost embedded solution capable of monitoring vital conditions and generating emergency alerts.

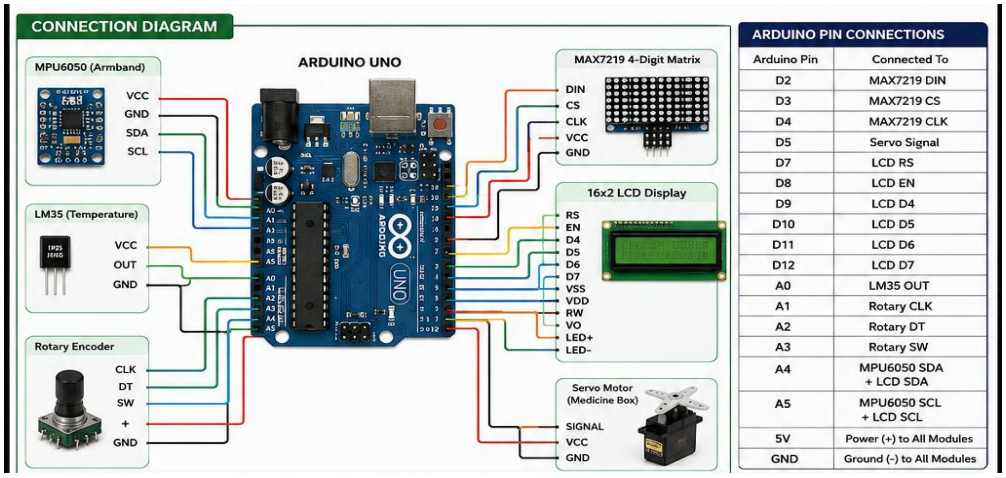
4.1.3 Objectives

- Monitor body temperature.
- Detect sudden falls using an MPU6050 sensor.
- Display health status on an LCD/OLED.
- Provide visual and audible alerts.
- Operate as a portable, low-cost health monitoring system.

4.1.4 Components Required

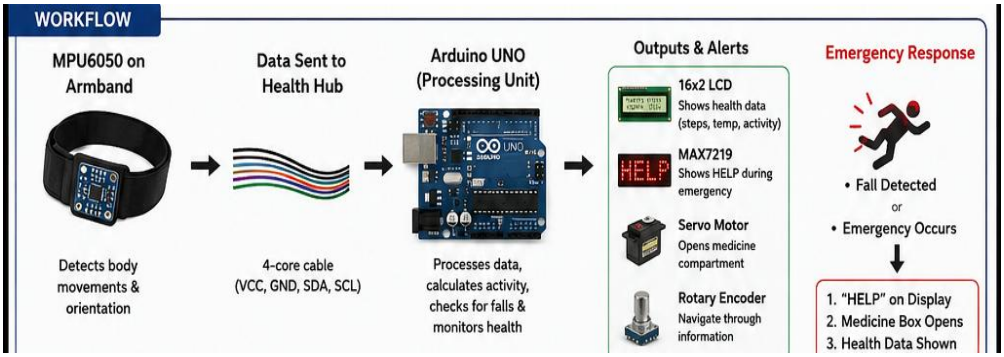
Component	Quantity	Purpose
Arduino UNO	1	Main controller
MPU6050	1	Fall detection
LM35 Temperature Sensor	1	Temperature monitoring
16×2 LCD / OLED	1	Display readings
Buzzer	1	Emergency alert
MAX7219 Display	1	Scrolling alert messages
Servo Motor	1	Demonstration of actuator response
Breadboard	1	Prototyping
Jumper Wires	As required	Connections
9V Battery	1	Power supply

4.1.5 System Architecture



4.1.6 Working Principle

1. The LM35 continuously measures the body temperature.
2. The MPU6050 monitors acceleration and angular motion.
3. The Arduino UNO processes the sensor data.
4. If abnormal conditions such as a fall or high temperature are detected, the buzzer is activated.
5. The LCD and MAX7219 display the health status and alert messages.
6. The system continues monitoring in real time until it is powered off.



4.1.7 Results

The prtotype successfully monitored temperature and detected abnormal motion. Alert messages were displayed correctly, and the buzzer responded immediately during emergency conditions. The system demonstrated reliable operation under repeated testing.

4.1.8 Advantages

- Low-cost design
- Portable
- Easy to operate
- Real-time monitoring
- Expandable with wireless communication

4.1.9 Limitations

- Prototype tested in a laboratory environment.
- Limited to selected sensors.
- Does not include cloud connectivity in the current version.

4.1.10 Future Scope

Future improvements may include:

- ESP32-based Wi-Fi connectivity
- Mobile application integration
- Cloud data storage
- Heart rate and SpO_2 monitoring

4.2 Mini Project 2: Fast Line Follower Robot Using PID Control

Objective - To develop an autonomous robot capable of accurately following a predefined path using the PID control algorithm.

Components Used

- Arduino UNO
- IR Sensor Array
- L298N Motor Driver
- DC Motors
- Robot Chassis
- Battery

Working Principle

The IR sensors detect the position of the line. The Arduino calculates the error and applies the PID algorithm to adjust the speed of the left and right motors, enabling smooth and accurate line following.

Flowchart

Start
  |
Read IR Sensors
  |
Calculate Error
  |
Apply PID
  |
Control Motors
  |
Repeat

Implementation

The robot was programmed using Arduino IDE. After tuning the PID parameters, it successfully followed straight and curved tracks with minimal oscillation.

Result

The robot achieved stable and accurate line following with improved speed and smooth turning.

4.3 Mini Project 3: Bluetooth-Controlled RC Car

Objective -To control a robotic car wirelessly using a smartphone through Bluetooth communication.

Components Used

- Arduino UNO
- HC-05 Bluetooth Module
- L298N Motor Driver
- DC Motors
- Battery

Working Principle

The smartphone sends movement commands via Bluetooth. The HC-05 receives these commands and the Arduino controls the motor driver to move the robot in different directions.

Flowchart

Start
  |
Connect Bluetooth
  |
Receive Command
  |
Control Motors
  |
Repeat

Implementation

Bluetooth communication was established between the Android application and the HC-05 module. The robot responded to Forward, Backward, Left, Right, and Stop commands.

Result

The RC car was successfully controlled wirelessly with smooth operation.

4.4 Mini Project 4: Aircraft Navigation Using MPU6050

Objective-To measure and monitor aircraft orientation using the MPU6050 accelerometer and gyroscope sensor.

Components Used

- Arduino UNO
- MPU6050
- OLED Display / Serial Monitor

Working Principle

The MPU6050 measures acceleration and angular velocity. Arduino processes this data to calculate Pitch, Roll, and Yaw, displaying the orientation in real time.

Flowchart

Start
  |
Read MPU6050
  |
Calculate Orientation
  |
Display Values
  |
Repeat

Implementation

The sensor was interfaced using the I²C protocol, and orientation values were displayed through the Serial Monitor.

Result

The system successfully measured real-time orientation for navigation applications.

CHAPTER 5:Major Project

Vision-Based Self-Driving Car Using Computer Vision and PID Control

This chapter should be the highlight of your report because it demonstrates the practical application of the concepts learned throughout the internship, including embedded systems, robotics, Linux, OpenCV, computer vision, control systems, and autonomous navigation.

5.1 Introduction

Autonomous vehicles are among the most significant innovations in modern robotics and artificial intelligence. A vision-based self-driving car uses computer vision algorithms, embedded controllers, and control systems to navigate roads without direct human intervention. Unlike traditional robotic vehicles that rely solely on sensors, a vision-based system uses a camera to detect lane markings, traffic signals, obstacles, and road conditions.

During this internship, a vision-based self-driving car was developed using **OpenCV** for image processing and a **PD/PID controller** for steering correction. The system captures real-time video from a camera, processes each frame to detect lanes, calculates the steering error, and generates control commands for autonomous navigation.

The project provided practical experience in image processing, robotics, control algorithms, and real-time decision-making, making it an excellent demonstration of modern autonomous vehicle technology.

5.2 Problem Statement

Road accidents caused by human error remain a major concern worldwide. Driver fatigue, distraction, and delayed reaction times contribute significantly to these accidents. Developing autonomous vehicles capable of detecting lanes, recognizing traffic signals, and maintaining stable steering can improve road safety and reduce dependence on human drivers.

The objective of this project was to design a prototype self-driving car capable of following road lanes autonomously using computer vision techniques and a PD/PID controller.

5.3 Objectives

- Develop a vision-based autonomous vehicle.
- Detect road lanes using OpenCV.
- Calculate steering error in real time.

- Implement PD/PID control for smooth steering.
- Detect traffic light signals.
- Achieve stable lane following with minimal oscillation.
- Understand the integration of computer vision and embedded systems.

5.4 System Requirements

Hardware Requirements

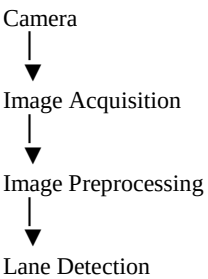
Component	Purpose
Camera	Capture road images
Embedded Controller	Vehicle control
Servo Motor	Steering control
DC Motor	Vehicle movement
Motor Driver	Motor control
Battery	Power supply
Robot Chassis	Mechanical platform

Software Requirements

Software	Purpose
Python	Programming language
OpenCV	Image processing
VS Code	Development environment
Linux	Development platform
NumPy	Numerical computation

5.5 System Architecture

The overall system consists of a camera, image-processing module, lane detection algorithm, steering controller, and motor control unit.



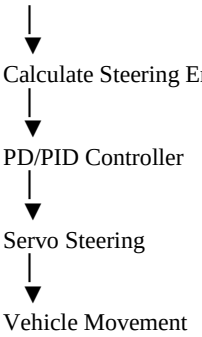


Figure 5.1: Overall System Architecture of the Vision-Based Self-Driving Car.

5.6 Working Principle

The camera continuously captures images of the road ahead. Each image is processed using OpenCV techniques such as grayscale conversion, Gaussian blur, edge detection, and region of interest extraction. The lane detection algorithm identifies the left and right lane boundaries and calculates the lane center.

The deviation between the lane center and the vehicle center is treated as the steering error. A PD/PID controller computes the appropriate steering correction, and the servo motor adjusts the vehicle direction accordingly. This process repeats continuously, enabling the vehicle to remain centered within the lane.

5.7 Image Processing Pipeline

The image-processing stages are as follows:

1. Capture image from the camera.
2. Convert the image to grayscale.
3. Apply Gaussian blur to reduce noise.
4. Perform Canny edge detection.
5. Define the Region of Interest (ROI).
6. Detect lane lines using the Hough Transform.
7. Calculate the lane center.
8. Compute steering error.
9. Apply PD/PID control.
10. Send steering commands to the servo motor.

5.8 Lane Detection Algorithm

The lane detection algorithm identifies road lane markings from each video frame.

Steps

1. Capture the image.
2. Convert RGB to grayscale.
3. Apply Gaussian filtering.
4. Perform Canny edge detection.
5. Mask the Region of Interest.
6. Detect lane lines using the Hough Line Transform.
7. Estimate lane center.
8. Calculate steering angle.

5.9 PD/PID Steering Controller

The PD/PID controller continuously minimizes the steering error by adjusting the servo motor.

Controller Equation

Output=Kp×Error+Ki×∫Error dt+Kd×d(Error)dtOutput = K_p \times Error + K_i \times \int Error\,dt + K_d \times \frac{d(Error)}{dt}Output=Kp×Error+Ki×∫Errordt+Kd×dtd(Error)

Where:

- **P** provides immediate correction.
- **I** removes accumulated error.
- **D** reduces overshoot and improves stability.

5.10 Traffic Light Detection

Traffic signals are detected using color segmentation techniques in the HSV color space.

Process

- Capture image.
- Convert to HSV.
- Detect red, yellow, and green color ranges.
- Identify the active traffic signal.
- Generate the corresponding vehicle action:
    - **Red:** Stop
    - **Yellow:** Slow Down
    - **Green:** Move Forward

5.11 Implementation

The project was implemented using Python and OpenCV. The lane detection algorithm processed each camera frame in real time, while the PD/PID controller generated steering corrections based on the calculated lane error. Traffic light detection was integrated to enable basic traffic signal recognition. The system was tested under controlled conditions to evaluate lane-following performance and steering stability.

5.12 Results

The prototype successfully detected lane boundaries and maintained lane alignment using the PD/PID controller. Traffic light detection functioned correctly under controlled lighting conditions. The vehicle demonstrated smooth steering corrections and stable navigation during testing.

5.13 Challenges Faced

- Variations in lighting conditions affected lane detection.
- Shadows and worn lane markings reduced detection accuracy.
- PID parameter tuning required multiple iterations.
- Camera calibration was necessary for improved performance.
- Real-time processing demanded efficient image optimization.

5.14 Applications

- Autonomous vehicles
- Advanced Driver Assistance Systems (ADAS)
- Warehouse automation
- Smart transportation
- Autonomous delivery robots
- Intelligent surveillance vehicles

5.15 Future Scope

- Integration of deep learning for object detection.
- GPS and LiDAR-based navigation.

- Traffic sign recognition.
- Obstacle avoidance using sensor fusion.
- Real-time cloud connectivity and remote monitoring.
- AI-based decision-making for complex driving environments.

CHAPTER 6

Skills Acquired

6.1 Introduction

The 45-day internship in **Embedded Systems and Robotics** provided valuable practical exposure to embedded programming, robotics, Linux, computer vision, and autonomous systems. It strengthened both technical knowledge and problem-solving abilities through hands-on implementation of real-world projects.

6.2 Technical Skills Acquired

During the internship, the following technical skills were developed:

Skill	Description
Embedded C	Programming microcontrollers
Arduino IDE	Embedded software development
ESP32 Programming	IoT applications
Python	Computer vision programming
Linux	Development environment
OpenCV	Image processing
PID Control	Autonomous robot control
Sensor Interfacing	Hardware integration
Bluetooth Communication	Wireless control
Motor Driver Programming	Robot movement

6.3 Software Tools Learned

Software/Tool	Purpose
Arduino IDE	Microcontroller programming
Visual Studio Code	Code development
Ubuntu Linux	Command-line practice and development
Python	Programming and automation
OpenCV	Computer vision and image processing
Git	Version control
Webots	Robotics simulation (if applicable)

CHAPTER 7

Conclusion

7.1 Conclusion

The **45-day internship in Embedded Systems and Robotics** provided an excellent opportunity to gain practical knowledge in embedded programming, robotics, Linux, computer vision, and autonomous systems. The internship successfully bridged the gap between classroom learning and real-world engineering applications.

Throughout the internship, theoretical concepts were reinforced through practical experiments and mini projects such as the **Smart Health Guardian**, **Fast Line Follower Robot**, **Bluetooth-Controlled RC Car**, **Aircraft Navigation using MPU6050**, and various **Computer Vision** applications.

The successful implementation of the **Vision-Based Self-Driving Car** as the major project demonstrated the integration of embedded systems, image processing, and control algorithms into a functional autonomous robotic system.

Overall, the internship enhanced technical competence, problem-solving ability, teamwork, and confidence in designing intelligent embedded solutions. The knowledge and experience gained during this training will serve as a strong foundation for future academic projects, research, and professional careers in robotics, embedded systems, artificial intelligence, and autonomous technologies.

7.2 Future Scope

The projects developed during the internship can be further enhanced by incorporating advanced technologies such as:

- Artificial Intelligence and Machine Learning
- Deep Learning-Based Object Detection
- LiDAR and GPS Integration
- Internet of Things (IoT)
- Cloud-Based Monitoring
- Autonomous Navigation
- Edge AI Computing
- Smart Healthcare Systems
- Industrial Automation
- Smart Transportation Solutions

REFERENCES

The following books, research papers, online resources, and software documentation were referred to during the internship and preparation of this report.

Books

1. Raj Kamal, **Embedded Systems: Architecture, Programming and Design**, McGraw Hill Education.
2. Muhammad Ali Mazidi, Janice Mazidi & Rolin McKinlay, **The AVR Microcontroller and Embedded Systems**, Pearson.
3. Richard Szeliski, **Computer Vision: Algorithms and Applications**, Springer.
4. John Morton, **Arduino for Everyone**, McGraw Hill.
5. Steven F. Barrett, **Embedded Systems Design with the Atmel AVR Microcontroller**, Morgan & Claypool.

Official Documentation

- Arduino Documentation
- ESP32 Documentation by Espressif Systems
- OpenCV Documentation
- Python Documentation
- Ubuntu Linux Documentation

Online Resources

- Arduino Project Hub
- GitHub Open-Source Repositories
- OpenCV Tutorials
- Linux Manual Pages (man)
- Stack Overflow (for troubleshooting and debugging)